

RCPSP Optimization API — Complete Route Documentation

Overview

This is a FastAPI-based REST API for solving **Resource-Constrained Project Scheduling Problems (RCPSP)** with multiple optimization objectives. The API uses PuLP with CBC solver to perform mathematical optimization on industrial pipeline scheduling scenarios.

Base URL: <http://localhost:8000>

Swagger UI: <http://localhost:8000/docs>

Table of Contents

1. [Health Check](#)
 2. [Problem Management \(CRUD\)](#)
 3. [Conflict Analysis](#)
 4. [Single-Objective Solving](#)
 5. [Pareto Front Optimization](#)
 6. [Weighted Aggregation Scalarization](#)
-

Health Check

GET [/health](#)

Purpose: Verify API availability and version.

Response:

```
{
  "status": "ok",
  "version": "2.0.0"
}
```

No calculations — simple health ping.

Problem Management (CRUD)

Problems represent RCPSP configurations including tasks, pipes, resources, and costs. The API maintains an in-memory store (no database).

GET [/problems](#)

Purpose: List all available problems with metadata.

Response:

```
[
  {
    "id": "default-pipeline-rcpsp",
    "name": "Default Pipeline RCPSP",
    "description": "Standard 3-zone refinery pipeline scenario",
    "n_zones": 3,
    "n_tasks": 15,
    "n_pipes": 4
  }
]
```

Calculations:

- Counts task IDs, pipe IDs, and zones from stored configurations
 - No optimization performed
-

GET /problems/default

Purpose: Bootstrap the frontend with a default problem configuration.

Response:

```
{
  "id": "default-pipeline-rcpsp",
  "name": "Default Pipeline RCPSP",
  "description": "...",
  "n_zones": 3,
  "n_tasks": 15,
  "n_pipes": 4,
  "task_ids": [1, 2, 3, ...],
  "pipes": ["Pipe A", "Pipe B", ...],
  "resources": [...],
  "task_costs": {...},
  "lag": {...},
  "alpha": {...},
  "dur_final": {...}
}
```

Calculations:

- Loads pre-configured default problem from `data/default_config.py`
 - No optimization
-

GET /problems/{problem_id}

Purpose: Fetch a specific problem by ID.

Path Parameters:

- `problem_id` (string): Unique problem identifier

Response: Same structure as `/problems/default`

Status Codes:

- `200 OK` — Problem found
 - `404 Not Found` — Problem does not exist
-

POST /problems

Purpose: Create a new problem configuration.

Request Body:

```
{
  "id": "my-custom-rcpsp",
  "name": "Custom Refinery Scenario",
  "description": "Custom 2-zone setup",
  "n_zones": 2,
  "task_ids": [1, 2, 3, 4, 5],
  "pipes": ["Pipe A", "Pipe B", "Pipe C"],
  "resources": [
    {
      "name": "Operator Team",
      "capacity": 10,
      "tasks": [
        { "task_id": 1, "consumption": 3 },
        { "task_id": 2, "consumption": 5 }
      ]
    }
  ],
  "task_costs": {
    "1": { "reinforced": 50, "normal": 0 }
  },
  "lag": {
    "1": { "0": 4, "1": 6 }
  },
  "alpha": {
    "1": 0.3,
    "2": 0.25
  },
  "dur_final": {
    "18": 48
  }
}
```

```
}  
}
```

Response:

```
{  
  "id": "my-custom-rcpsp",  
  "name": "Custom Refinery Scenario"  
}
```

Status Code: 201 Created

PUT /problems/{problem_id}

Purpose: Update an existing problem.

Path Parameters:

- `problem_id` (string): Problem to update

Request Body: Same as POST

Response:

```
{  
  "id": "my-custom-rcpsp",  
  "name": "Custom Refinery Scenario"  
}
```

Status Code: 200 OK or 404 Not Found

DELETE /problems/{problem_id}

Purpose: Remove a problem configuration.

Path Parameters:

- `problem_id` (string): Problem to delete

Status Codes:

- 204 No Content — Successfully deleted
 - 400 Bad Request — Cannot delete the default problem
 - 404 Not Found — Problem does not exist
-

Conflict Analysis

POST /conflicts

Purpose: Detect and analyze **resource conflicts** in a problem.

What are Resource Conflicts?

A conflict occurs when multiple tasks require the same resource simultaneously and their combined consumption exceeds the resource's capacity. This analysis identifies which task pairs conflict and cannot run in parallel.

Request Body:

```
{
  "config": {
    "id": "my-problem",
    "name": "...",
    "n_zones": 2,
    ...
  }
}
```

Response:

```
{
  "conflicts": [
    {
      "resource": "Operator Team",
      "capacity": 10,
      "task_waiting": 3,
      "task_blocking": 1,
      "task_waiting_name": "Distillation",
      "task_blocking_name": "Preprocessing"
    }
  ],
  "resources": [
    {
      "name": "Operator Team",
      "capacity": 10,
      "tasks": [
        {
          "task_id": 1,
          "consumption": 3,
          "task_name": "Preprocessing"
        }
      ]
    }
  ]
}
```

```
"n_conflicts": 1
}
```

Calculations Performed:

1. **Conflict Detection Algorithm** (`detect_conflicts`):
 - Sorts all tasks by index
 - Accumulates resource consumption in order
 - When accumulated consumption would exceed capacity:
 - Flags the waiting task and the blocking task as conflicting
 - Resets accumulation for the next group
2. **Resource Summaries**:
 - Lists each resource with its capacity
 - Groups tasks consuming each resource
 - Maps task IDs to human-readable names

Use Case: Understand resource bottlenecks before optimization.

Single-Objective Solving

`POST /solve`

Purpose: Find an **optimal schedule** for one of two objectives:

- **Makespan** (minimize project duration)
- **Cost** (minimize total reinforcement cost)

Request Body:

```
{
  "config": {
    "id": "my-problem",
    ...
  },
  "objective": "makespan"
}
```

Response:

```
{
  "objective": "makespan",
  "status": "optimal",
  "cmax_zone_h": 168.5,
  "cmax_total_h": 850.2,
  "cmax_total_days": 35.4,
  "total_cost_da": 1450,
}
```

```

"reinforced_count": 5,
"starts": {
  "1": {"0": 0, "1": 12.5, "2": 25.3, "3": 38.1},
  "2": {"0": 4.2, "1": 16.7, ...}
},
"modes": {
  "1": {"0": 0, "1": 1, "2": 0, "3": 1},
  "2": {"0": 1, "1": 0, ...}
},
"ganttt_bars": [
  {
    "task_id": 1,
    "task_name": "Preprocessing",
    "pipe": 0,
    "pipe_name": "Pipe A",
    "zone": 1,
    "start": 0.0,
    "duration": 12.5,
    "end": 12.5,
    "reinforced": false
  }
],
"ganttt_summary": {
  "total_hours": 850.2,
  "total_days": 35.4,
  "total_bars": 60
},
"config_id": "my-problem",
"config_name": "..."
}

```

Calculations Performed:

MILP Model Construction

1. Decision Variables:

- $d[\text{task}][\text{pipe}]$: Start time of each task on each pipe (continuous, ≥ 0)
- $y[\text{task}][\text{pipe}]$: Binary reinforcement flag (0=normal duration, 1=reinforced with 30% reduction)
- $C_{\max}$: Project makespan per zone (continuous, ≥ 0)

2. Objective Function:

- **Makespan**: Minimize $C_{\max}$
- **Cost**: Minimize $\text{sum}(\text{cost_reinforced}[i][j] * y[i][j]) * N_ZONES + \text{fixed_costs}$

3. Constraints:

a) Makespan Definition:

```
Cmax >= d[last_task][pipe] + effective_duration(last_task, pipe)
for all pipes
```

b) Precedence (Task Sequencing):

```
d[next_task][pipe] >= d[current_task][pipe] + LAG[current_task][pipe]
```

Ensures tasks on the same pipe respect precedence with inter-task lag.

c) Inter-Pipe Sequencing:

```
d[task][next_pipe] >= d[task][pipe] + effective_duration(task, pipe)
```

Tasks must finish on one pipe before starting on the next.

d) Resource Conflict Constraints:

```
d[waiting_task][pipe_0] >= d[blocking_task][last_pipe] +
eff_dur(blocking_task, last_pipe)
```

Prevents simultaneous execution of conflicting tasks.

4. Effective Duration Calculation:

```
eff_dur(task, pipe) = DUR[task][pipe] * (1 - ALPHA[task] * y[task][pipe])
```

- $\text{ALPHA}[\text{task}]$ = reduction factor (e.g., 0.3 = 30% faster if reinforced)
- When reinforced ($y=1$): duration decreases
- When normal ($y=0$): full duration

Solution Extraction

- Solves MILP with **CBC (Coin-or-Branch-and-Cut)** solver
- Extracts:
 - Start times (starts)
 - Reinforcement decisions (modes)
 - Zone makespan and total project duration
 - Reinforcement cost across all zones

Gantt Chart Generation

- Converts solver schedule into Gantt bars with:
 - Absolute project time offsets per zone
 - Inter-zone lag of 76 hours (INTER_ZONE_LAG)
 - Task names, pipe names, reinforcement flags
 - Final sequential tasks appended after zones

Status Codes:

- **200 OK** — Optimal solution found
 - **422 Unprocessable Entity** — Solver found no optimal solution
 - **400 Bad Request** — Invalid objective value
-

Pareto Front Optimization

POST /pareto

Purpose: Generate a **Pareto front** between two conflicting objectives:

- **F1:** Minimize makespan (project duration)
- **F2:** Minimize cost (reinforcement expenses)

The API uses the **ϵ -constraint method** to sweep between F1* (pure makespan optimization) and F2* (pure cost optimization), discovering all non-dominated solutions.

Request Body:

```
{
  "config": {
    "id": "my-problem",
    ...
  },
  "n_points": 12
}
```

Parameters:

- **n_points:** Number of Pareto points to generate (3–30, default 12)

Response:

```
{
  "status": "ok",
  "message": "12 Pareto points found.",
  "f1": {
    "objective": "F1_Makespan",
    "status": "optimal",
    "cmax_zone_h": 150.0,
    "cmax_total_h": 810.0,
  }
}
```

```

    "cmax_total_days": 33.75,
    "total_cost_da": 3200,
    "reinforced_count": 12,
    "ganttBars": [...]
  },
  "f2": {
    "objective": "F2_Cost",
    "status": "optimal",
    "cmax_zone_h": 200.0,
    "cmax_total_h": 960.0,
    "cmax_total_days": 40.0,
    "total_cost_da": 500,
    "reinforced_count": 0,
    "ganttBars": [...]
  },
  "pareto_points": [
    {
      "epsilon": 150.0,
      "cmax_zone_h": 150.0,
      "cmax_total_h": 810.0,
      "cmax_total_days": 33.75,
      "total_cost_da": 3200,
      "reinforced_count": 12
    },
    {
      "epsilon": 160.0,
      "cmax_zone_h": 160.0,
      "cmax_total_h": 858.0,
      "cmax_total_days": 35.75,
      "total_cost_da": 2400,
      "reinforced_count": 8
    },
    ...
  ],
  "n_requested": 12,
  "config_id": "my-problem",
  "config_name": "..."
}

```

Calculations Performed:

Phase 1: Anchor Solutions

1. Solve F1 (Pure Makespan Minimization):

- Build MILP: minimize C_{max}
- Provides: $\epsilon_{ps_min} = F1*.cmax_zone_h$
- Result: Maximum reinforcement cost, minimum duration

2. Solve F2 (Pure Cost Minimization):

- Build MILP: minimize `sum(costs) * N_ZONES`
- Provides: `eps_max = F2*.cmax_zone_h`
- Result: Minimal or no reinforcement, longer duration

Phase 2: ϵ -Constraint Sweep

For each epsilon in `linspace(eps_min, eps_max, n_points)`:

1. Build MILP with:
 - **Constraint:** `Cmax <= epsilon` (force makespan to decreasing values)
 - **Objective:** Minimize total cost
2. Solve and record:
 - Duration achieved
 - Cost at that duration
 - Number of reinforced tasks
3. Results form a **trade-off curve**: as duration increases, cost decreases

Gantt Charts

- Attaches complete Gantt bar schedules to both F1* and F2* anchor solutions

Status Codes:

- **200 OK** — Pareto front generated
- **422 Unprocessable Entity** — Could not solve F1* or F2*

Use Case: Help decision-makers choose between speed vs. cost trade-offs.

Weighted Aggregation Scalarization

POST `/weighted`

Purpose: Generate **weighted Pareto-optimal solutions** using a continuous scalarization approach.

Unlike ϵ -constraint (which solves discrete points), weighted aggregation smoothly varies a weighting parameter $\lambda \in [0,1]$ and solves:

$$\text{minimize } \lambda \cdot \text{F2} + (1 - \lambda) \cdot \text{F1}$$

Where:

- `F1_norm = (total_time - min_time) / (max_time - min_time)` (normalized makespan)
- `F2_norm = (total_cost - min_cost) / (max_cost - min_cost)` (normalized cost)

Request Body:

```
{
  "config": {
    "id": "my-problem",
    ...
  }
}
```

```
},  
  "n_lambdas": 15  
}
```

Parameters:

- **n_lambdas**: Number of λ points to evaluate (3–30, default 15)

Response:

```
{  
  "status": "ok",  
  "n_lambdas": 15,  
  "bounds": {  
    "temps_min": 150.0,  
    "temps_max": 200.0,  
    "cout_min": 500,  
    "cout_max": 3200  
  },  
  "f1": {  
    "objective": "F1_Makespan",  
    "cmax_zone_h": 150.0,  
    "cmax_total_h": 810.0,  
    "cmax_total_days": 33.75,  
    "total_cost_da": 3200,  
    "reinforced_count": 12  
  },  
  "f2": {  
    "objective": "F2_Cost",  
    "cmax_zone_h": 200.0,  
    "cmax_total_h": 960.0,  
    "cmax_total_days": 40.0,  
    "total_cost_da": 500,  
    "reinforced_count": 0  
  },  
  "results": [  
    {  
      "lambda": 0.0,  
      "status": "optimal",  
      "cmax_total_h": 810.0,  
      "cmax_total_days": 33.75,  
      "total_cost_da": 3200,  
      "duration_norm": 0.0,  
      "cost_norm": 1.0,  
      "reinforced_count": 12  
    },  
    {  
      "lambda": 0.0714,  
      "status": "optimal",  
      "cmax_total_h": 825.0,  
      "cmax_total_days": 34.375,  
      "total_cost_da": 500,  
      "duration_norm": 0.0,  
      "cost_norm": 1.0,  
      "reinforced_count": 12  
    }  
  ]  
}
```

```

        "total_cost_da": 2800,
        "duration_norm": 0.075,
        "cost_norm": 0.826,
        "reinforced_count": 10
    },
    ...
    {
        "lambda": 1.0,
        "status": "optimal",
        "cmax_total_h": 960.0,
        "cmax_total_days": 40.0,
        "total_cost_da": 500,
        "duration_norm": 1.0,
        "cost_norm": 0.0,
        "reinforced_count": 0
    }
],
"unique_solutions": [
    {
        "lambda_values": [0.0, 0.0714],
        "cmax_total_h": 810.0,
        "cmax_total_days": 33.75,
        "total_cost_da": 3200,
        "duration_norm": 0.0,
        "cost_norm": 1.0,
        "reinforced_count": 12,
        "status": "optimal"
    },
    ...
]
}

```

Calculations Performed:

Step 1: Determine Bounds

- Solve $F1^*$ (minimize makespan) $\rightarrow$ temps_min, cout_max
- Solve $F2^*$ (minimize cost) $\rightarrow$ temps_max, cout_min

Step 2: Normalization

For each λ value, compute:

```

denom_time = max(temps_max - temps_min, 1)
denom_cost = max(cout_max - cout_min, 1)

F1_norm = (total_time - temps_min) / denom_time
F2_norm = (total_cost - cout_min) / denom_cost

```

Step 3: Weighted Minimization

For each λ in `linspace(0, 1, n_lambdas)`:

1. Build MILP with objective:

```
minimize  $\lambda * F2\_norm + (1-\lambda) * F1\_norm$ 
```

2. When $\lambda = 0$: Pure time minimization (identical to F1)
3. When $\lambda = 1$: Pure cost minimization (identical to F2)
4. When $0 < \lambda < 1$: Weighted trade-off

Step 4: Deduplication

- Some λ values may produce identical solutions (rounded to nearest hour/100 DA)
- Aggregates duplicate solutions and lists all λ values that yield them
- `unique_solutions` contains only non-dominated results

Status Codes:

- `200 OK` — Weighted solutions generated
- `422 Unprocessable Entity` — Could not determine F1* or F2* bounds

Use Case: Explore smooth trade-offs and discover optimal compromise solutions.

Key Concepts

Makespan

- **Definition:** Duration of one optimization cycle (in hours)
- **For one zone:** `cmax_zone_h` (e.g., 168.5 hours)
- **For all zones:** `cmax_total_h = cmax_zone_h * N_ZONES + final_task_duration`

Reinforcement

- **What:** Hiring additional personnel to accelerate tasks
- **Cost:** Specified in `task_costs["reinforced"]` per task and zone
- **Effect:** Multiplies duration by `(1 - ALPHA[task])`
 - Example: ALPHA=0.3 means 30% time reduction
 - Normal: 100 hours → Reinforced: 70 hours

Resource Conflicts

- **Definition:** Two tasks cannot run in parallel due to shared resource
- **Example:** Both tasks require "Operator Team" (capacity 10), but combined consumption is 15
- **Resolution:** Solver ensures one task waits for the other to finish

Effective Duration

$$\text{eff_dur}(i,j) = \text{DUR}[i][j] \times (1 - \text{ALPHA}[i] \times y[i][j])$$

- If reinforced ($y=1$): reduced duration
- If normal ($y=0$): full duration

Gantt Chart

- **Visual schedule** showing all tasks across all pipes and zones
- **Time Offset:** Zone k starts at $(k-1) * (\text{cmax_zone} + \text{INTER_ZONE_LAG})$
- **INTER_ZONE_LAG:** 76 hours between zone starts

Error Responses

All errors return standard HTTP status codes with descriptive messages:

```
{
  "detail": "Problem not found."
}
```

Status	Meaning
200	Success
201	Created
204	Deleted (no content)
400	Bad request (invalid input)
404	Not found
422	Solver failed or invalid constraints

Example Workflow

1. Bootstrap

```
curl -X GET http://localhost:8000/problems/default
```

Retrieve default problem configuration.

2. Analyze Conflicts

```
curl -X POST http://localhost:8000/conflicts \  
-H "Content-Type: application/json" \  
-d '{"config": {...}}'
```

Understand resource bottlenecks.

3. Solve for Makespan

```
curl -X POST http://localhost:8000/solve \  
-H "Content-Type: application/json" \  
-d '{"config": {...}, "objective": "makespan"}'
```

Find fastest schedule.

4. Generate Pareto Front

```
curl -X POST http://localhost:8000/pareto \  
-H "Content-Type: application/json" \  
-d '{"config": {...}, "n_points": 12}'
```

Explore speed vs. cost trade-offs.

5. Weighted Exploration

```
curl -X POST http://localhost:8000/weighted \  
-H "Content-Type: application/json" \  
-d '{"config": {...}, "n_lambdas": 15}'
```

Find compromise solutions with smooth weighting.

Performance Notes

- **Solve time:** 1–10 seconds for typical problems (15 tasks, 3–4 pipes)
- **Pareto generation:** 30–60 seconds for 12 points (12 sequential MILP solves)
- **Weighted aggregation:** 20–40 seconds for 15 λ values
- Solver: CBC (free, open-source, reliable for MILP)

Technical References

- **MILP:** Mixed Integer Linear Programming
- **CBC:** Coin-or-Branch-and-Cut solver (PuLP backend)

- **ϵ -Constraint:** Pareto front generation via sequential optimization
- **Weighted Aggregation:** Scalarization using convex combinations

See solver code in [solver/](#) directory for implementation details.